

项目创新说明

赛题假定减少 RVV 向量状态保存与恢复是优化吞吐和延迟的关键。本项目先完成 V 状态开销的可观测量化，并据此证伪“向量保存是主要瓶颈”；再在 Linux sched_ext 上实现自定义调度器，用长时间片、AI 核合理分配、推理线程无感知识别和推理保护窗口完成 TTFT 与 TPS 优化。

1. V 状态瓶颈证伪：可观测量化

1.1. 最接近的已有工作

Linux 在任务切换路径上对 Dirty V 做惰性保存（`riscv_v_vstate_save` 一类接口），以及赛题建议的在该保存点上挂探针统计调用。惰性保存见 `docs/scheduler/RISC-V名词解析与调度概述.md`。保存点不可直接挂载的结论见 `docs/tracker/bpftrace使用调研.md`。

1.2. 已有方案具备的能力

内核只会在 VS 为 Dirty 时写入向量寄存器，Clean / Off 则跳过。这解决的是“要不要保存”，不回答“保存了多少次、每次切换多贵、是否拖累同一次推理的 TPS/TTFT”。

直接在保存函数上挂 kprobe 行不通：`riscv_v_vstate_save` 为 `static __always_inline`，`/proc/kallsyms` 中没有符号；路径上的 `__switch_to` 带 `notrace`，`bpftrace` 无法附着。已有工作停在不可 trace 的 inline 保存点。

1.3. 本项目新增的技术贡献

本项目提出绕过该保存点的量化方案：用已有 `sched:sched_switch` tracepoint 从内核栈读取 `pt_regs.status` 的 VS 域（`0x0600` 为 Dirty），统计 save/restore 频次，并按进程、CPU、线程分布；再用 `kprobe:finish_task_switch` 度量从切换起点到切换完成的耗时，区分 V 状态切换与非 V 状态切换，从而同时量化频次与开销。该方案不依赖未导出的 `riscv_vstate_save`。

把上述指标与同一次运行的 TPS、TTFT 对照后得到结论：推理线程已经占住目标核时，V 状态保存次数不是吞吐的代理指标；有干扰时吞吐下降，损失的是推理线程让出核心后的计算时间。优化应转向减少有效计算被打断，而不是单纯压低保存次数。

1.4. 代码位置与验证依据

项	位置
频次：全局 save/restore	<code>src/tracker/vstate_trace.bt</code>
频次：按 CPU、线程分布	<code>src/tracker/vstate_trace_cpu.bt</code>

项	位置
开销：V / 非 V 切换耗时	src/tracker/vstate_trace_llama_server.bt （ sched_switch 记起点， finish_task_switch 记终点）
与 TPS、TTFT 同轮采集	src/benchmark/bench_runner.py 、 src/benchmark/bench_runner_all.py 、 src/benchmark/bench_runner_final.py

tests/test/group6/：3B 模型、4 线程、核 4–7。taskset.csv 三轮保存约 4.1 万次，聚合吞吐约 2.86；taskset-chrt-f.csv 保存约 6300 次，吞吐约 2.72。保存次数降到约六分之一，吞吐没有上升。

tests/test/group3/ 同样是保存明显下降、吞吐略降：taskset.csv 保存约 5.8 万次、聚合吞吐约 1.92；taskset-chrt-f.csv 保存约 9600 次、吞吐约 1.86。

2. 自定义调度器：长时间片、AI 核分配、无感知识别与保护窗口

2.1. 最接近的已有工作

默认调度器 CFS；用户态绑核手段 taskset、cgroup cpuset 等，见 docs/tracker/CPU_affinity_cgroup.md；可编程调度框架 Linux sched_ext（select_cpu / enqueue / dispatch 与 DSQ），见 src/scheduler/自定义调度.md。

2.2. 已有方案具备的能力

CFS 按公平份额分配 CPU，不区分矩阵乘 worker 与网络、组 batch 线程，也不感知 K1 上带 IME 的 cluster0 与通用 cluster1。

taskset 与 cgroup 按进程限制可用核，llama-server 内各类线程会绑在同一组核上；partition-root 可以双向独占核组，仍不按线程角色分流。

sched_ext 提供用 BPF 替换 CFS 选核与入队的能力，并可在卸载后回退 CFS。框架本身不含推理线程识别规则，也不规定长时间片、AI 核放置或矩阵乘期间的抢占保护。

2.3. 本项目新增的技术贡献

在 sched_ext 上实现自定义调度器 vec_affine_sched，完成面向 TTFT / TPS 的优化。相对上述已有工作，新增四点：

- 推理线程无感知识别：**用 uprobe / uretprobe 挂在 libggml-cpu.so 的 ggml_compute_forward_mul_mat 上，把执行过矩阵乘的线程标为推理 worker。不改 llama.cpp，也不重编内核。
- AI 核合理分配：**推理 worker 放在 CPU 0–3（K1 cluster0，带 IME），其余任务放在 CPU 4–7（cluster1）；原核空闲时优先回到上次运行的核。

- 3. **长时间片**：推理线程 30ms，普通任务 2ms；推理线程被唤醒时插入本核队列头部。
- 4. **推理保护窗口**：线程处于 mul_mat 计算内部且未超过推理时间片时，tick 跳过抢占，避免矩阵乘被打断。

2.4. 代码位置与验证依据

项	位置
无感知识别 (uprobe 入口 / 出口)	src/scheduler/vec_affine_sched.bpf.c 的 vecsched_mul_mat_entry、vecsched_mul_mat_exit；附着在 src/scheduler/vec_affine_sched.c
AI 核选核与原核 优先	同 BPF 文件的 vec_select_cpu
长时间片与唤醒 插队	vec_enqueue；SLICE_INFER_NS 为 30ms，SLICE_DEFAULT_NS 为 2ms
推理保护窗口	vec_tick：in_mul_mat 且未超时则 STAT_PROTECT_WINDOW 并返回

tests/sched_ext/：Llama-3.2-3B，20 请求同时发出，prompt 64 token，输出 20 token，每组 3 轮。无额外干扰时，CFS 聚合吞吐 6.81，本方案 (2ms / 30ms) 6.88。高负载下 CFS 为 1.76，仅换 sched_ext、时间片仍为 4ms/20ms 时为 1.86，与 CFS 接近；采用 2ms/30ms 后为 3.34，吞吐约提高 90%，平均 TTFT 从 4298ms 降到 2880ms。高负载为 cpu_stress 占 CPU 4-7，见 tests/sched_ext/README.md。